BINV2150-A — Javascript : avancé

Séance 22 - Session et web storage

Haute École Léonard de Vinci

Rappel workflow - session utilisateur

- Le frontend contacte le backend à une route `/login`
- Le backend génère un JWT sécurisé et le renvoie au frontend
- Le frontend stocke de manière sécurisée ce JWT
- Pour chaque requête devant être identifiée, le frontend envoie ce JWT dans le header d'authorisation
- Le backend reçoit le JWT avec chaque requête et vérifie la signature pour autoriser l'accès

Session frontend

- Session : période de temps où une activité est en cours
- Entre le chargement du site et un rechargement ou sa fermeture
- L'application retient les valeurs de ses variables d'état

Exemple - login/logout

```tsx
// App.tsx
const [token, setToken] = useState<string | undefined>(undefined)

const login = async (credentials: Credentials): Promise<boolean> => {
  const response = await fetch('/api/auths/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', },
    body: JSON.stringify(credentials),
  })
  if (!response.ok) return false

  const data = await response.json()
  setToken(data.token)
  return true
}

const logout = () => {
  setToken(undefined)
  navigate('/login')
}
```

Exemple - Utilisation du token

```tsx
// App.tsx
const isAuthenticated = token !== undefined

const getProfile = async (): Promise<User | undefined> => {
  if (!token) throw new Error('Not authenticated')

  const response = await fetch('/api/auths/profile', {
    headers: { 'Authorization': token },
  });

  if (!response.ok) return undefined;

  const user = await response.json();
  return user;
}
```

Web Storage API

- API du navigateur pour stocker des données côté client
- 2 types de stockage :
  - **localStorage** : persistant (même après fermeture du navigateur)
  - **sessionStorage** : temporaire (supprimé à la fermeture de l'onglet)

```
// Stocker une donnée
localStorage.setItem("item", data);

// Récupérer une donnée
const data = localStorage.getItem("item");

// Supprimer une donnée
localStorage.removeItem("item");

// Tout effacer
localStorage.clear();
```

```
// Stocker une donnée
sessionStorage.setItem("item", data);

// Récupérer une donnée
const data = sessionStorage.getItem("item");

// Supprimer une donnée
sessionStorage.removeItem("item");

// Tout effacer
sessionStorage.clear();
```

Récupérer une session

```tsx
// App.tsx
const [token, setToken] = useState<string | undefined>(undefined)

useEffect(() => {
  const savedToken = localStorage.getItem("token");
  if (savedToken) setToken(savedToken);
}, [])

const login = async (credentials: Credentials): Promise<boolean> => {
  // ...
  const data = await response.json()
  setToken(data.token)
  localStorage.setItem("token", data.token);
  return true
}

const logout = () => {
  setToken(undefined)
  localStorage.removeItem("token");
  navigate('/login')
}
```